

# Ingeniería de Software II

---

2026 – CLASE 4

# Contenidos

---

- ❖ Gestión de Proyectos
  - Planificación Temporal
- ❖ Diseño de Software - Conceptos

# Gestión de Proyectos

## Planificación Temporal

La planificación temporal es una actividad que distribuye el esfuerzo estimado a lo largo de la duración prevista del proyecto.

La precisión es muy importante para no generar:

- clientes insatisfechos,
- costos adicionales,
- reducción del impacto en el mercado,
- ...

# Calendarización

---

- ❖ La ***calendarización del proyecto*** de software es una acción que distribuye el esfuerzo estimado a través de la duración planificada del proyecto, asignando el esfuerzo a ***tareas*** específicas del desarrollo de software.

# Calendarización

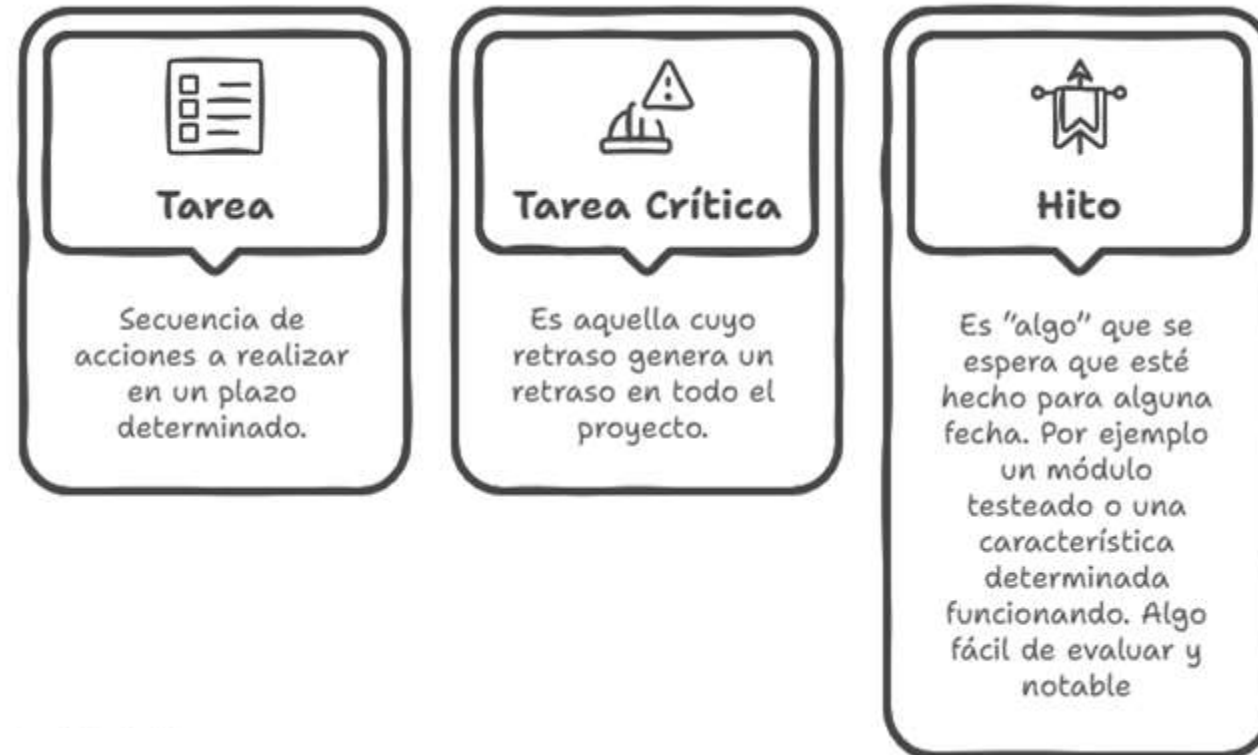
- ❖ Las fechas de los proyectos pueden ser de dos tipos:



- ❖ La fecha impuesta por el cliente suele ser la más frecuente.

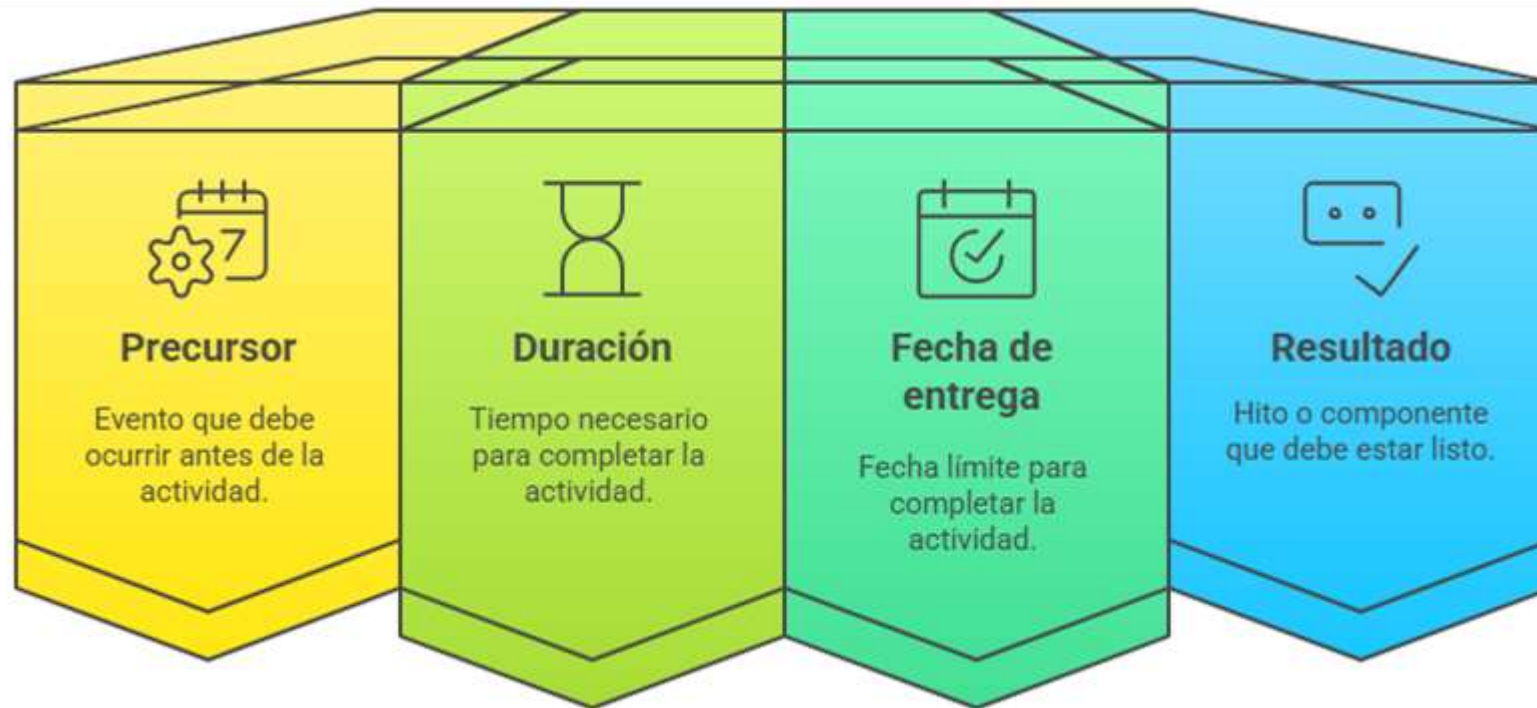
# Calendarización

❖ La Planificación Temporal está compuesta por:



# Calendarización

❖ Una tarea se describe por 4 parámetros:



# Calendarización - Red de tareas

---

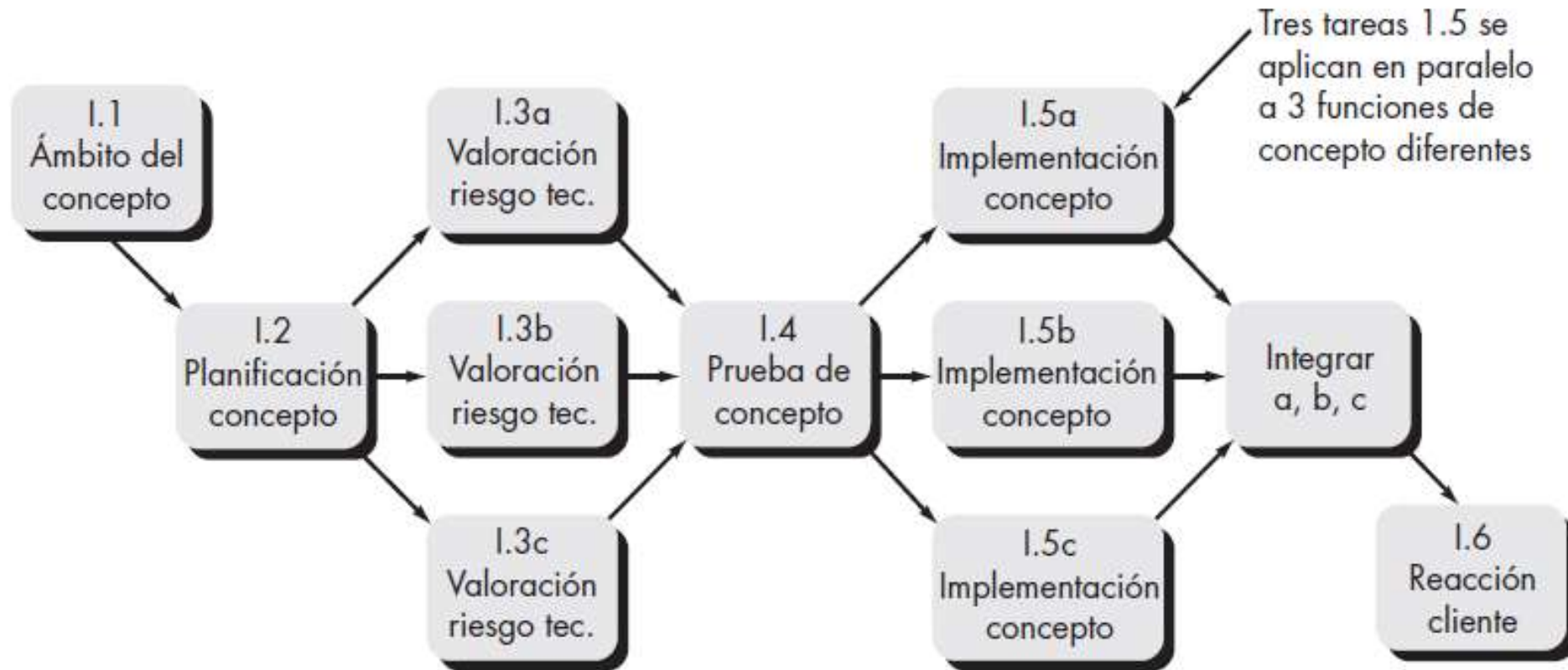
- ❖ Es una representación gráfica del flujo de las tareas desde el inicio hasta el fin de un proyecto.
- ❖ En algunos casos los conjuntos de tareas permiten realizar algunas actividades en paralelo.
- ❖ Representan la secuencia de las tareas y su interdependencia.

# Calendarización - Red de tareas

---

- ❖ El conjunto de tareas variará dependiendo del tipo de proyecto y grado de rigor.
  
- ❖ Los factores que influyen en el conjunto de tareas a elegir son :
  - Tamaño del proyecto
  - Número de usuarios potenciales
  - Criticidad del proyecto
  - Estabilidad de los requerimientos
  - Facilidad de comunicación con el cliente/usuario
  - Madurez de la tecnología aplicable
  - Restricciones
  - ...

# Calendarización - Red de tareas



# Planificación Temporal – Diagrama de Gantt

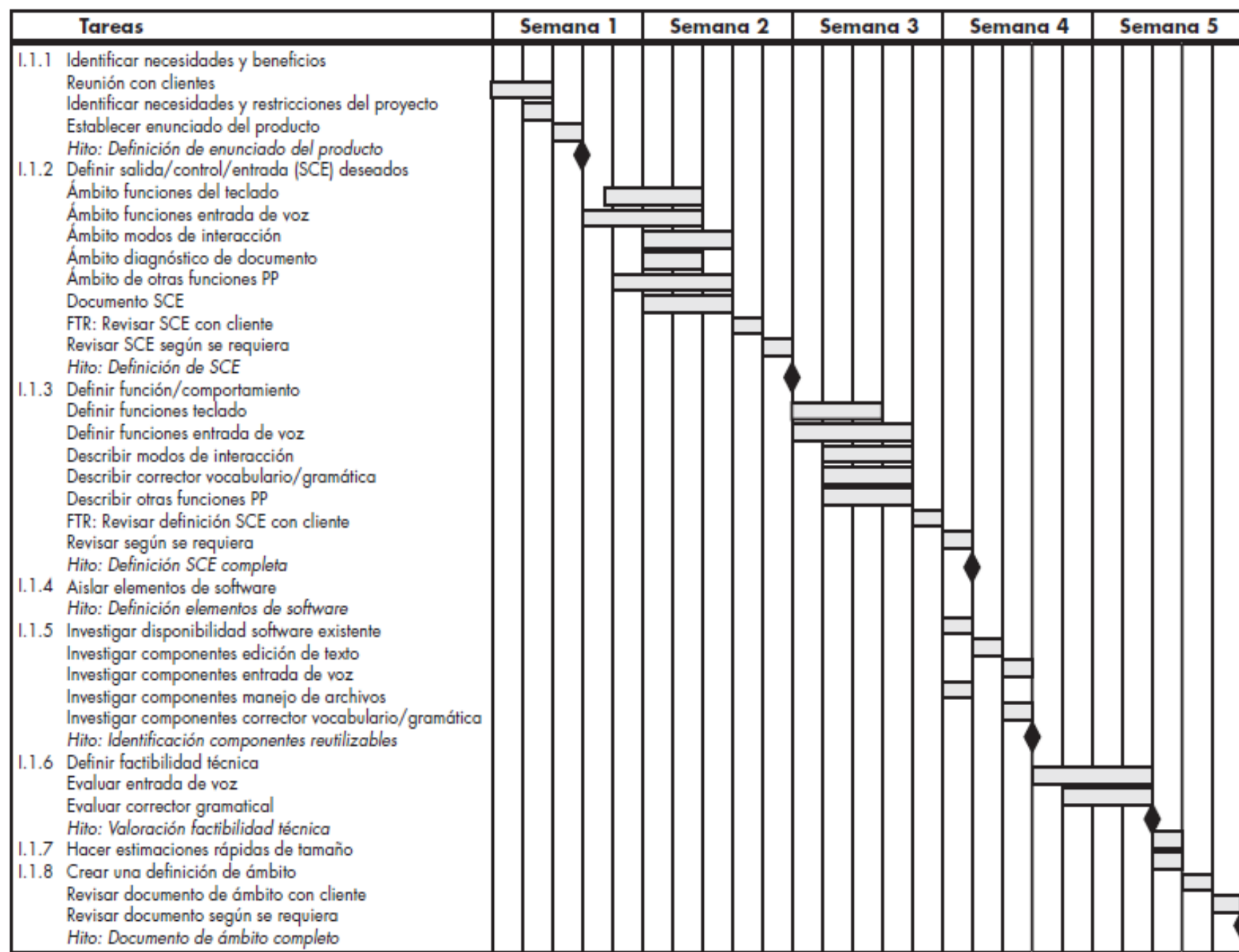
---

- ❖ Un diagrama de Gantt es una herramienta visual de gestión de proyectos utilizada para planificar y rastrear el progreso de diversas tareas y actividades dentro del ciclo de vida de un proyecto de software.
- ❖ Funciona como una línea de tiempo visual, ofreciendo una visión general de alto nivel de los cronogramas del proyecto, lo que simplifica la gestión de planes complejos que involucran múltiples equipos y plazos cambiantes.

# Planificación Temporal – Diagrama de Gantt

## ❖ Componentes:





# Planificación Temporal – PERT y CPM

- ❖ **PERT** (Técnica de Evaluación y Revisión de Programas) es una herramienta utilizada para analizar las estimaciones de tareas en un cronograma y evaluar las opciones de la ruta crítica, especialmente cuando las duraciones de las tareas son inciertas.
- ❖ **CPM** (Método de la Ruta Crítica) es un método que identifica la secuencia de actividades que determinan el tiempo mínimo de finalización de un proyecto, centrándose en tareas bien definidas con duraciones conocidas.
- ❖ Ambos ayudan a desglosar proyectos complejos en tareas manejables, visualizar cronogramas, identificar dependencias, etc., apuntando a la finalización del proyecto a tiempo.

# Planificación Temporal – PERT y CPM

---

## **PERT** (Program Evaluation & Review Technique):

- ❖ Creado para proyectos del programa de defensa del gobierno norteamericano entre 1958 y 1959.
- ❖ Se utiliza para controlar la ejecución de proyectos con gran número de actividades que implican investigación, desarrollo y pruebas.
- ❖ Red de tareas con Fechas tempranas, tardías, Camino crítico.
- ❖ *Probabilístico.*

# Planificación Temporal – PERT y CPM

---

## CPM (Critical Path Method):

- ❖ Desarrollado para dos empresas americanas entre 1956 y 1958.
- ❖ Se utiliza en proyectos en los que hay poca incertidumbre en las estimaciones.
- ❖ Tiempo de inicio temprano y tardío.
- ❖ *Determinístico.*

# Planificación Temporal – PERT y CPM

❖ Actualmente se ha tomado lo mejor de ambos métodos y se han vuelto uno solo, conocido como ***Método del Camino Crítico***.

1. Establecer lista de tareas
2. Fijar dependencia entre tareas y duración
3. Construir la red
4. Numerar los nodos
5. Calcular la fecha temprana y tardía de cada nodo  
     $Te_i$  = Fecha temprana del nodo  $i$   
     $Tai$  = Fecha tardía del nodo  $i$
6. Calcular el camino crítico que une las tareas críticas  
     $\Rightarrow Te_i = Tai$

# Planificación Temporal – PERT y CPM

## ❖ Método del Camino Crítico – Ejemplo:

1. Establecer lista de tareas
2. Fijar dependencia entre tareas y duración

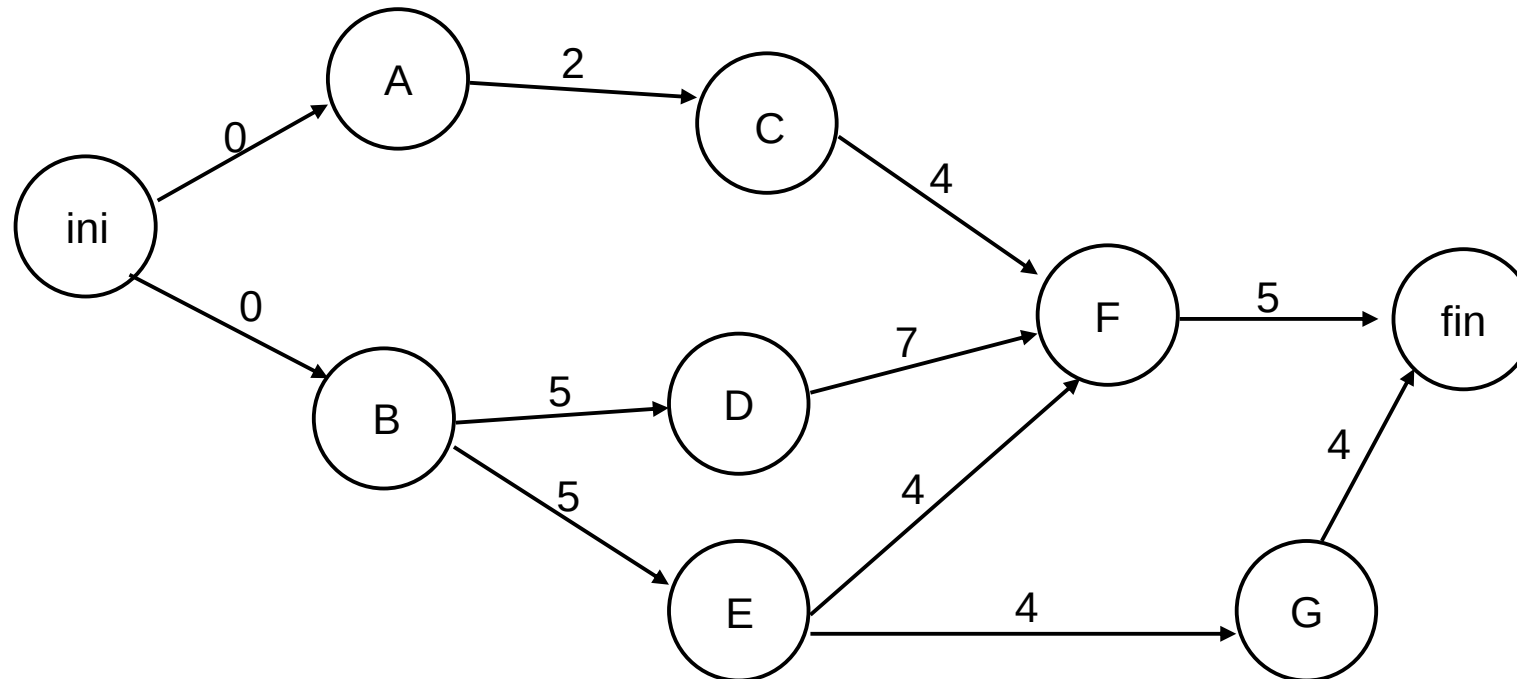
| Tarea | Precedida por | Duración |
|-------|---------------|----------|
| A     | -             | 2        |
| B     | -             | 5        |
| C     | A             | 4        |
| D     | B             | 7        |
| E     | B             | 4        |
| F     | C-D-E         | 5        |
| G     | E             | 4        |

# Planificación Temporal – PERT y CPM

## ❖ Método del Camino Crítico – Ejemplo:

3. Construir la red

4. Numerar los nodos



# Planificación Temporal – PERT y CPM

## ❖ Método del Camino Crítico – Ejemplo:

5. Calcular la fecha temprana y tardía de cada nodo

Fechas Tempranas:

$$TeJ = Tel + tIJ$$

Donde:

$TeJ$  = fecha más temprana del nodo destino

$Tel$  = fecha más temprana del nodo origen

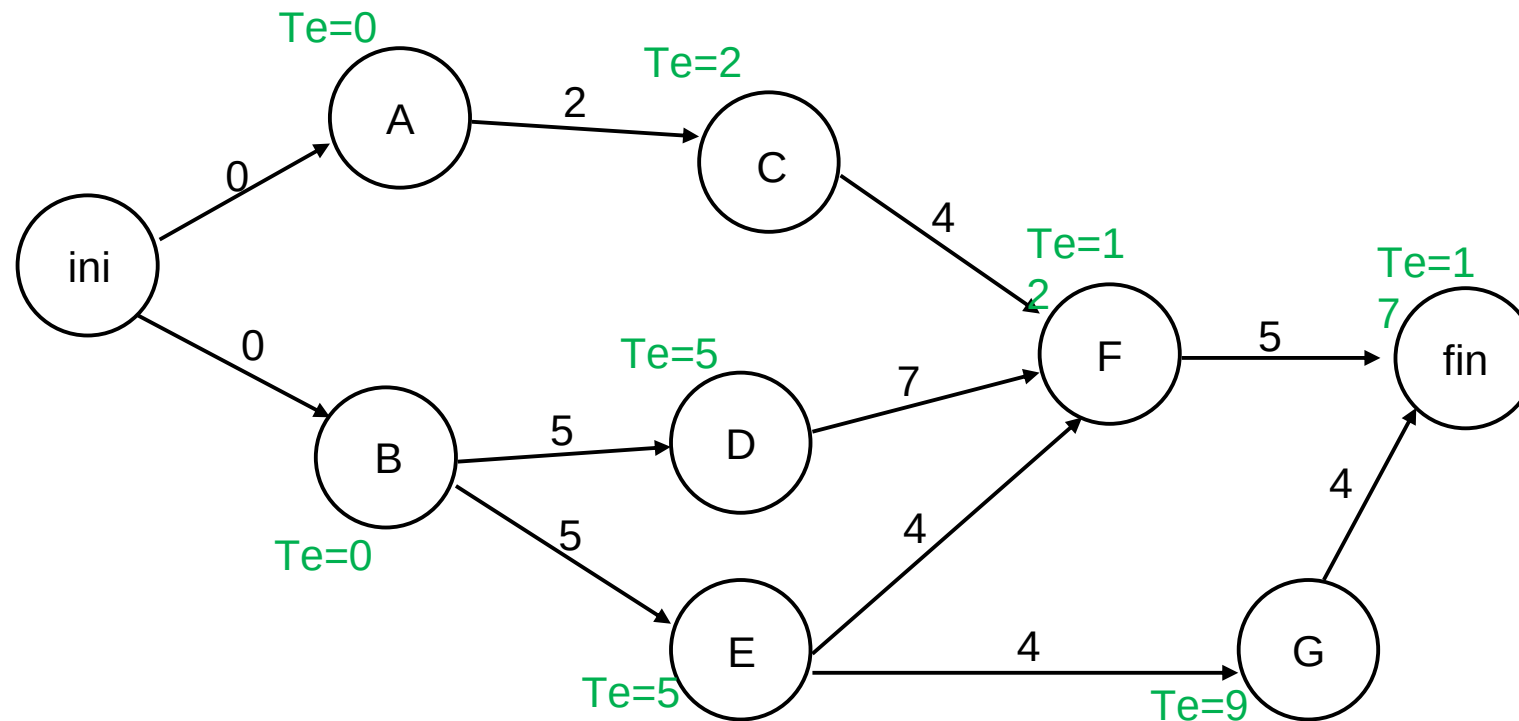
$tIJ$  = duración de la tarea desde el nodo I hasta el nodo J

Si hay más de un camino:  $\text{Max}(TeJ1, TeJ2..)$

| Tarea | Precedida por | Duración |
|-------|---------------|----------|
| A     | -             | 2        |
| B     | -             | 5        |
| C     | A             | 4        |
| D     | B             | 7        |

# Planificación Temporal – PERT y CPM

- ❖ Método del Camino Crítico – Ejemplo:
- 5. Calcular la fecha temprana y tardía de cada nodo



# Planificación Temporal – PERT y CPM

## ❖ Método del Camino Crítico – Ejemplo:

5. Calcular la fecha temprana y tardía de cada nodo

### Fechas Tardías:

$$TaI = TaJ - tIJ$$

Donde:

$TaI$  = fecha más tardía del nodo origen

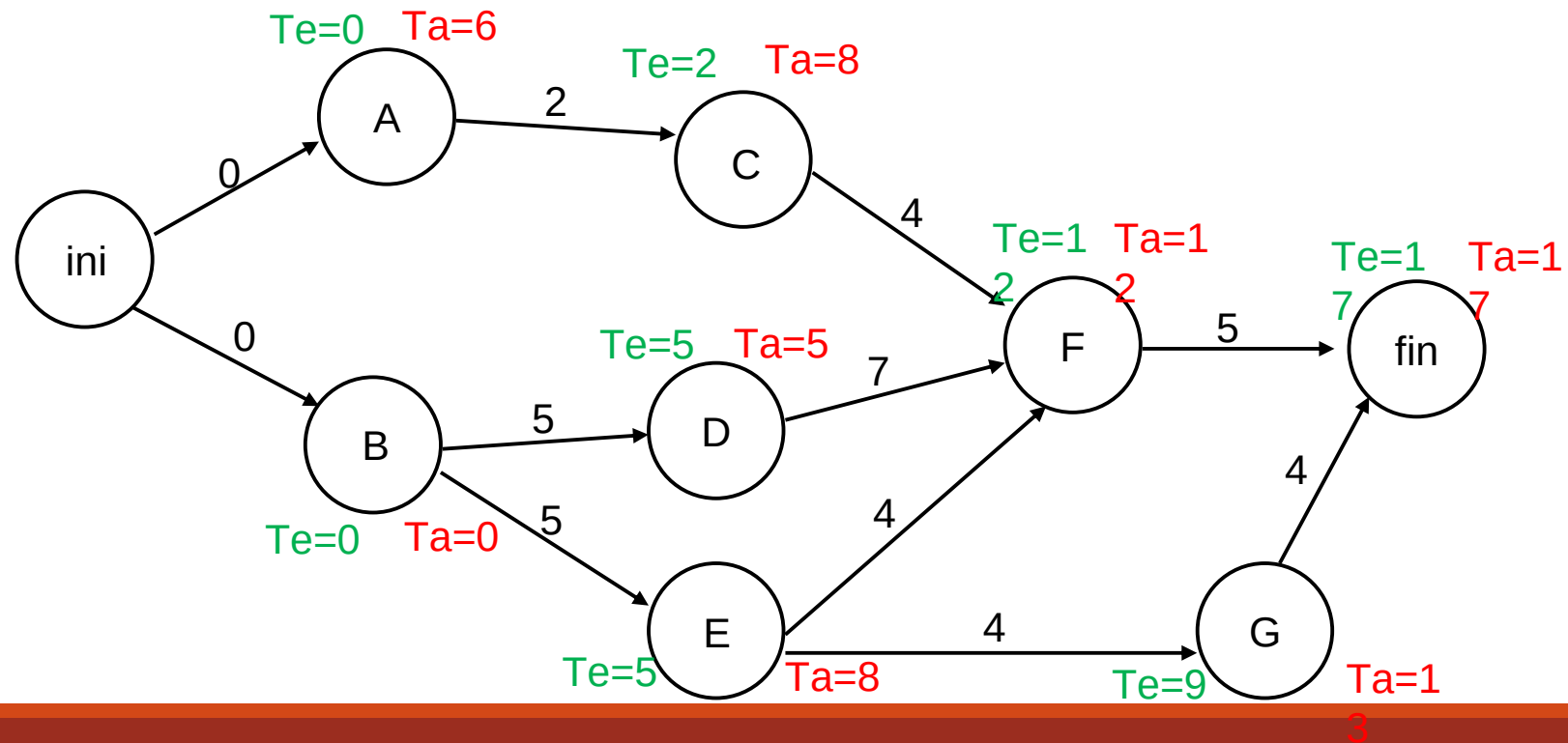
$TaJ$  = fecha más tardía del nodo destino

$tIJ$  = duración de la tarea desde el nodo I hasta el nodo J

Si hay más de un camino:  $\text{Min}(TaJ1, TaJ2..)$

# Planificación Temporal – PERT y CPM

- ❖ Método del Camino Crítico – Ejemplo:
5. Calcular la fecha temprana y tardía de cada nodo



# Planificación Temporal – PERT y CPM

## ❖ Método del Camino Crítico – Ejemplo:

6. Calcular el camino crítico que une las tareas críticas

Márgen Total:

$$Mt = TaJ - Tel - tIJ$$

Donde:

$TaJ$  = fecha tardía del nodo destino

$Tel$  = fecha temprana del nodo origen

$tIJ$  = duración de la tarea desde el nodo I hasta el nodo J

*Observar que el Margen Total también puede calcularse como  $MtJ = TaJ - Tel$*

*Es decir, como la diferencia entre la fecha más tardía y más temprana del mismo nodo.*

# Planificación Temporal – PERT y CPM

---

## ❖ Método del Camino Crítico – Ejemplo:

6. Calcular el camino crítico que une las tareas críticas

¿Qué ocurre cuando tengo un margen total de, por ej., 6 días?

*Significa que la tarea puede iniciarse con 6 días de retraso sin que ello afecte a la duración total del proyecto.*

# Planificación Temporal – PERT y CPM

---

## ❖ Método del Camino Crítico – Ejemplo:

6. Calcular el camino crítico que une las tareas críticas

¿Qué ocurre cuando el margen total es 0?

*Significa que no hay margen y que esa tarea hay que iniciarla y finalizarla en las fechas más tempranas.*

*Estas tareas con margen cero serían críticas.*

# Planificación Temporal – PERT y CPM

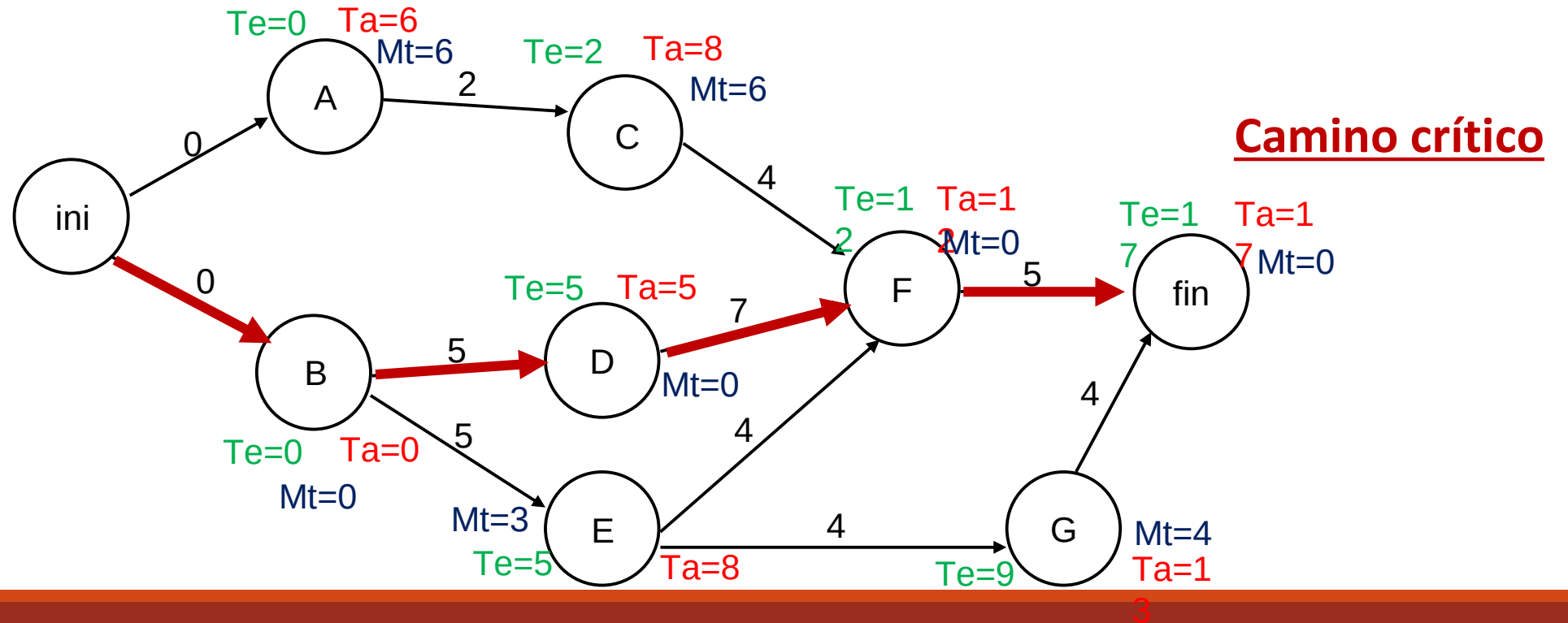
---

- ❖ Método del Camino Crítico – Ejemplo:
  - 6. Calcular el camino crítico que une las tareas críticas
    - El camino formado por una sucesión de tareas críticas recibe el nombre de camino crítico.
    - El camino crítico puede obtenerse utilizando el cálculo del Margen Total.

# Planificación Temporal – PERT y CPM

## ❖ Método del Camino Crítico – Ejemplo:

6. Calcular el camino crítico que une las tareas críticas



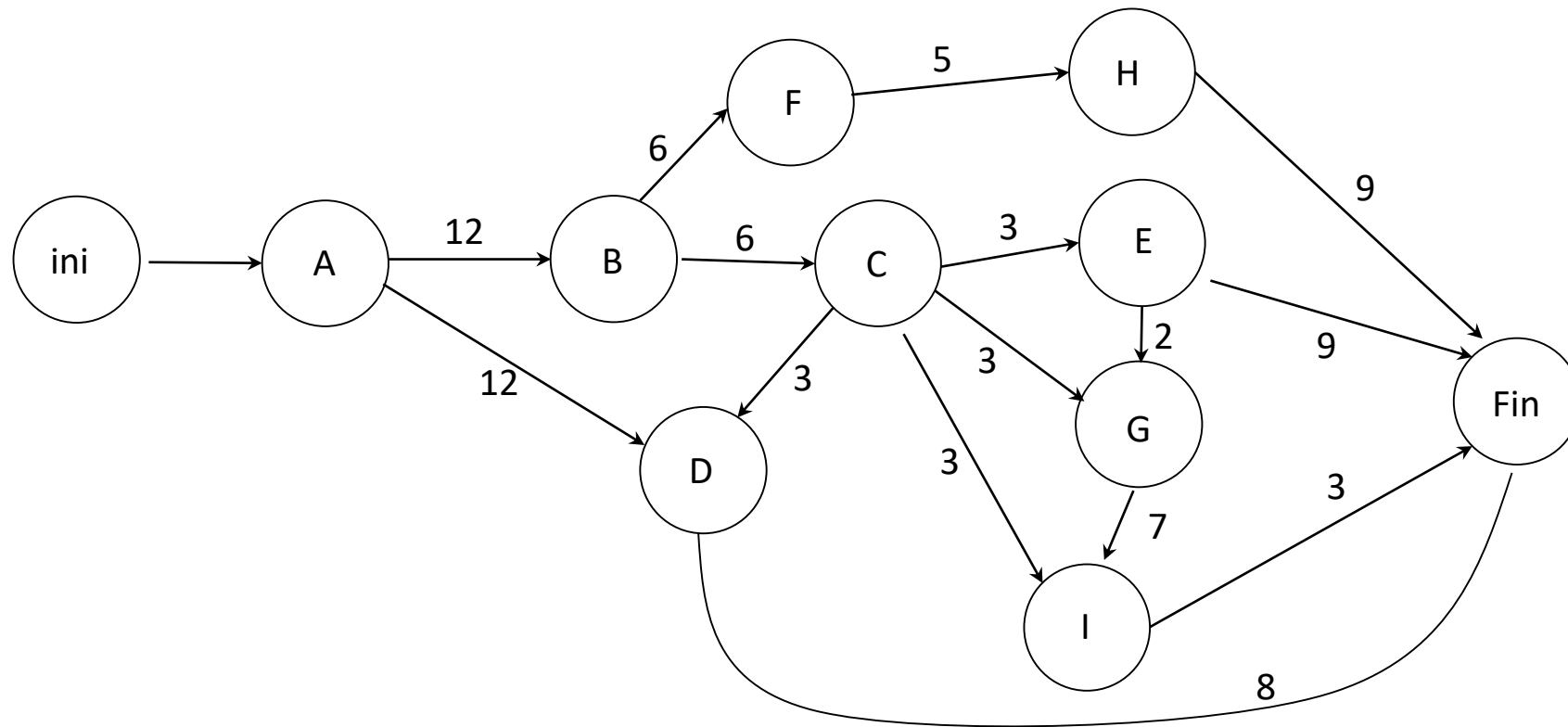
# Planificación Temporal – PERT y CPM

## ❖ Método del Camino Crítico – Otro ejemplo:

| Tarea | Duración | Restricciones                                            |
|-------|----------|----------------------------------------------------------|
| A     | 12       | -                                                        |
| B     | 5        | A terminada                                              |
| C     | 3        | Empieza 1 semana después de terminada B                  |
| D     | 8        | A terminada, C terminada                                 |
| E     | 9        | C terminada                                              |
| F     | 6        | Empieza 6 semanas después del comienzo de B              |
| G     | 4        | C terminada. Empieza 2 semanas después del comienzo de E |
| H     | 9        | Empieza 1 semana antes del fin de F                      |
| I     | 3        | C terminada. Empieza 3 semanas después del fin de G      |

# Planificación Temporal – PERT y CPM

❖ Método del Camino Crítico – Otro ejemplo:



# Planificación Temporal – PERT y CPM

## ❖ Método del Camino Crítico – Otro ejemplo:

| Tarea | Duración | Restricciones                                            | Te | Ta |
|-------|----------|----------------------------------------------------------|----|----|
| A     | 12       | -                                                        | 0  | 0  |
| B     | 5        | A terminada                                              | 12 | 12 |
| C     | 3        | Empieza 1 semana después de terminada B                  | 18 | 18 |
| D     | 8        | A terminada, C terminada                                 | 21 | 25 |
| E     | 9        | C terminada                                              | 21 | 21 |
| F     | 6        | Empieza 6 semanas después del comienzo de B              | 18 | 19 |
| G     | 4        | C terminada. Empieza 2 semanas después del comienzo de E | 23 | 23 |
| H     | 9        | Empieza 1 semana antes del fin de F                      | 23 | 24 |
| I     | 3        | C terminada. Empieza 3 semanas después del fin de G      | 30 | 30 |
| Fin   | 0        | -                                                        | 33 | 33 |

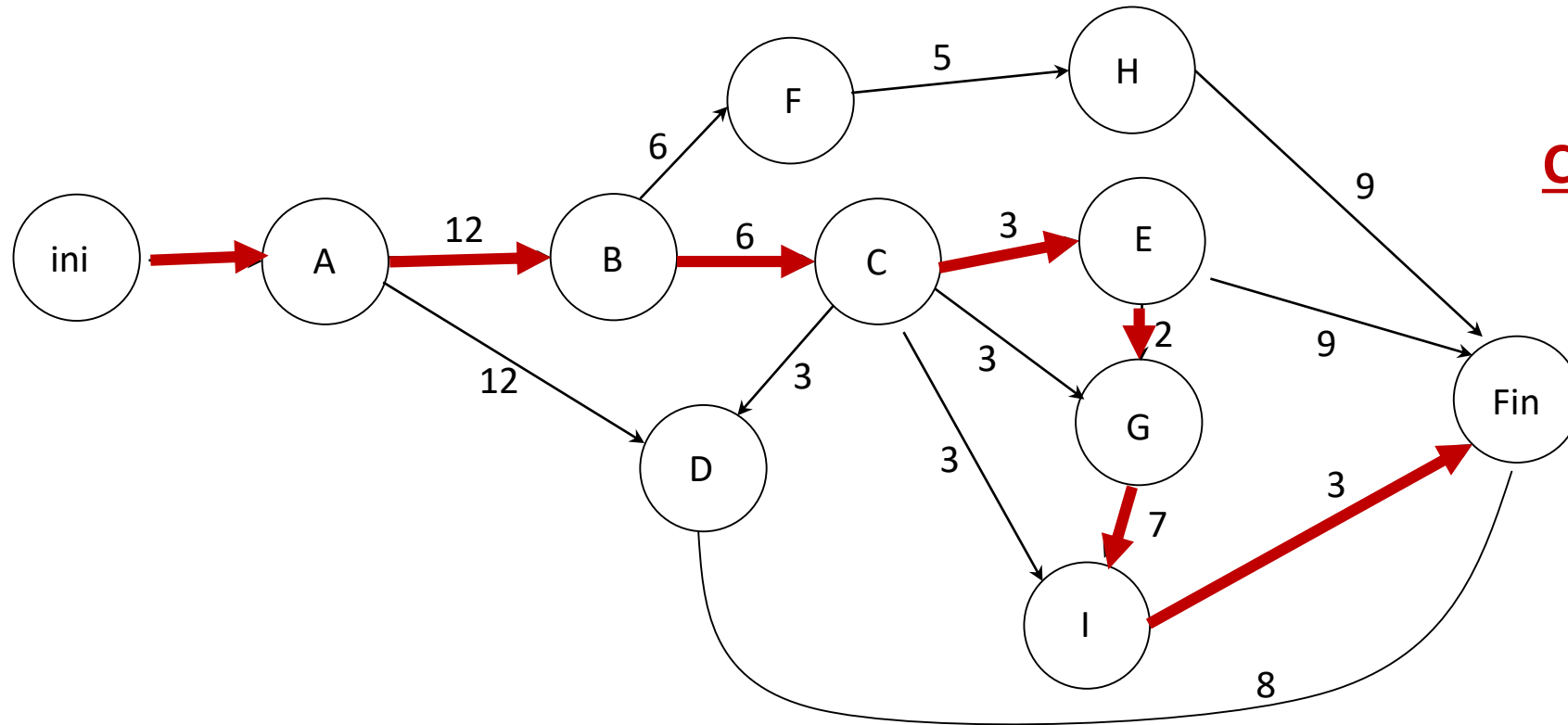
# Planificación Temporal – PERT y CPM

## ❖ Método del Camino Crítico – Otro ejemplo:

| Tarea | Duración | Restricciones                                            | Te | Ta |
|-------|----------|----------------------------------------------------------|----|----|
| A     | 12       | -                                                        | 0  | 0  |
| B     | 5        | A terminada                                              | 12 | 12 |
| C     | 3        | Empieza 1 semana después de terminada B                  | 18 | 18 |
| D     | 8        | A terminada, C terminada                                 | 21 | 25 |
| E     | 9        | C terminada                                              | 21 | 21 |
| F     | 6        | Empieza 6 semanas después del comienzo de B              | 18 | 19 |
| G     | 4        | C terminada. Empieza 2 semanas después del comienzo de E | 23 | 23 |
| H     | 9        | Empieza 1 semana antes del fin de F                      | 23 | 24 |
| I     | 3        | C terminada. Empieza 3 semanas después del fin de G      | 30 | 30 |
| Fin   | 0        | -                                                        | 33 | 33 |

# Planificación Temporal – PERT y CPM

❖ Método del Camino Crítico – Otro ejemplo:



**Camino crítico**

# Planificación Temporal – PERT y CPM

❖ Datos que se obtienen:



# Planificación Temporal

---

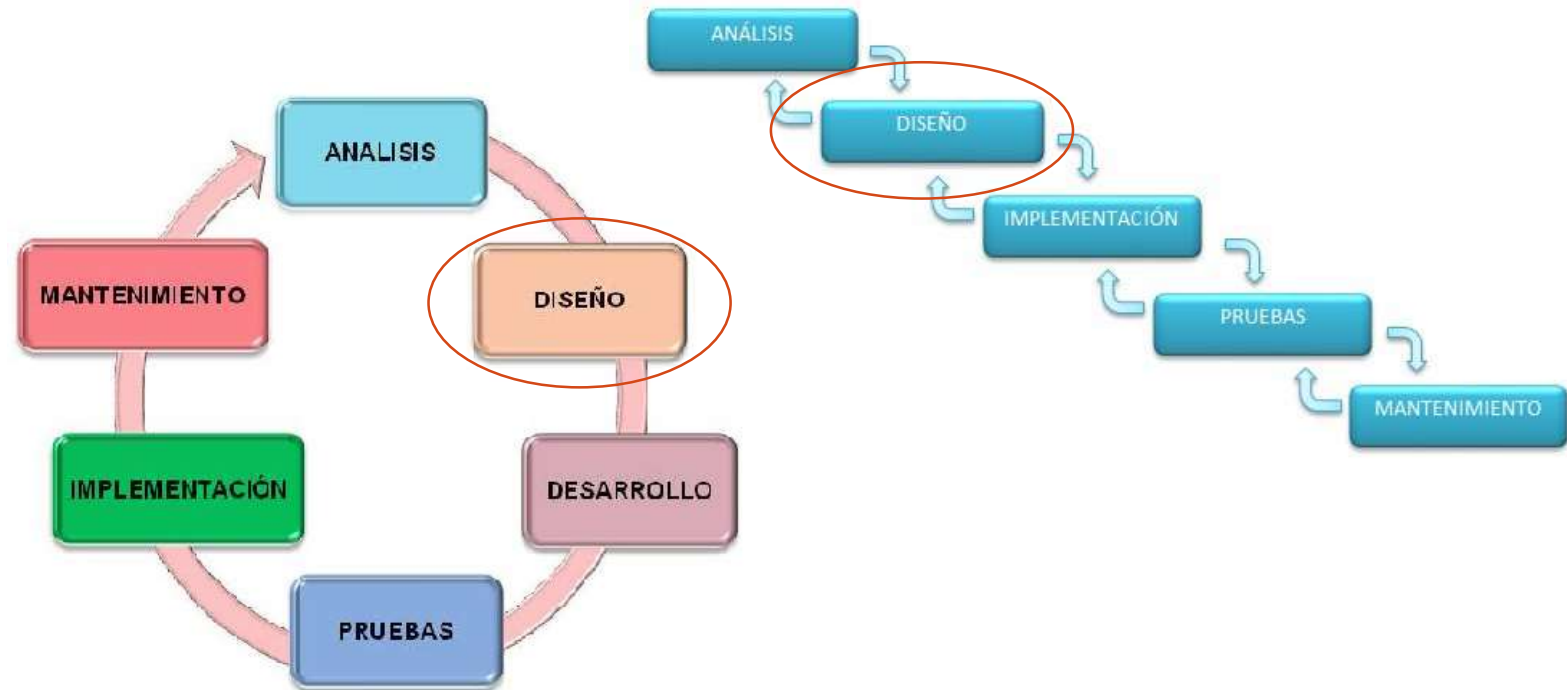
## ❖ ¿Qué hacer cuando una tarea se sale de la agenda?

- Revisar el impacto sobre la fecha de entrega
- Reasignar recursos

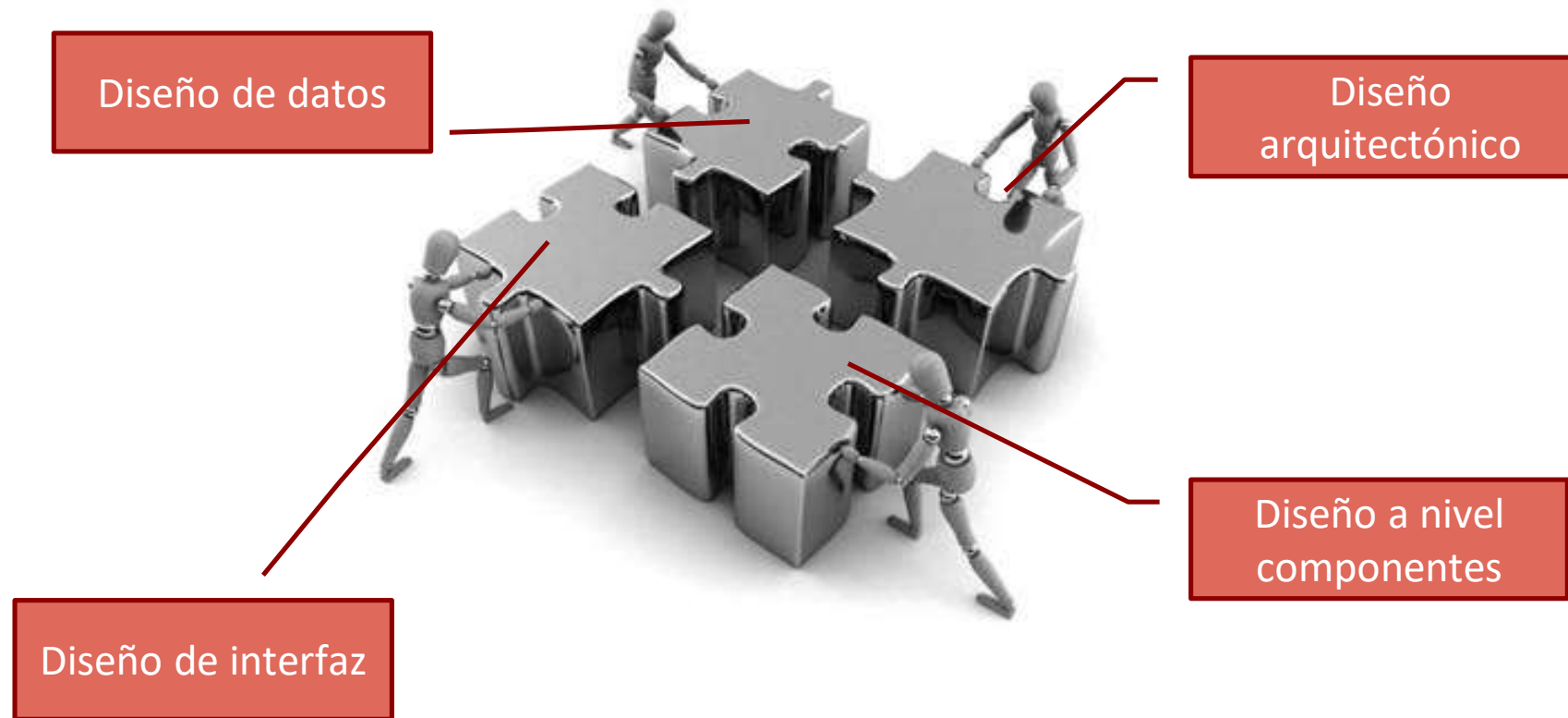
*La inclusión de más personas en el desarrollo no siempre genera aumento en la productividad*

- Reordenar tareas
- Modificar entrega

# Conceptos



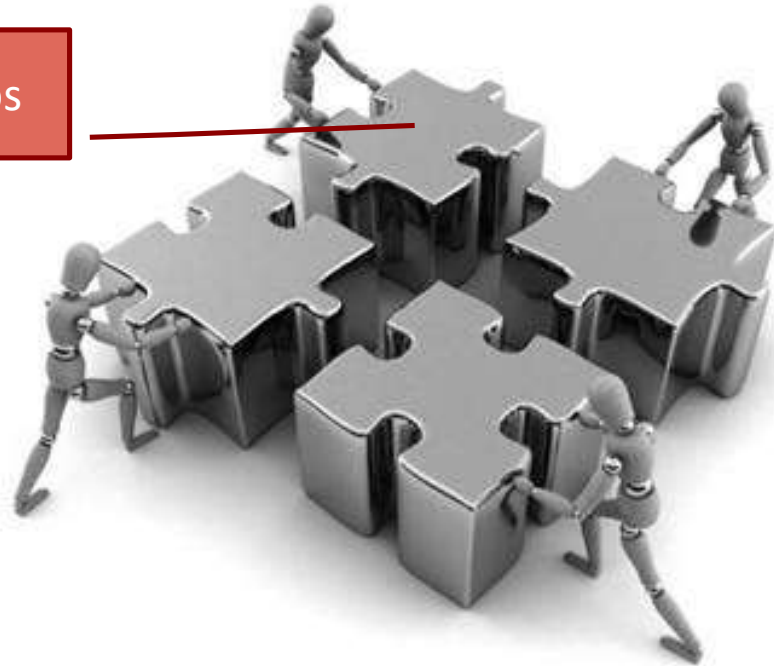
# Diseño de Software - Tipos



# Diseño de Software - Tipos

---

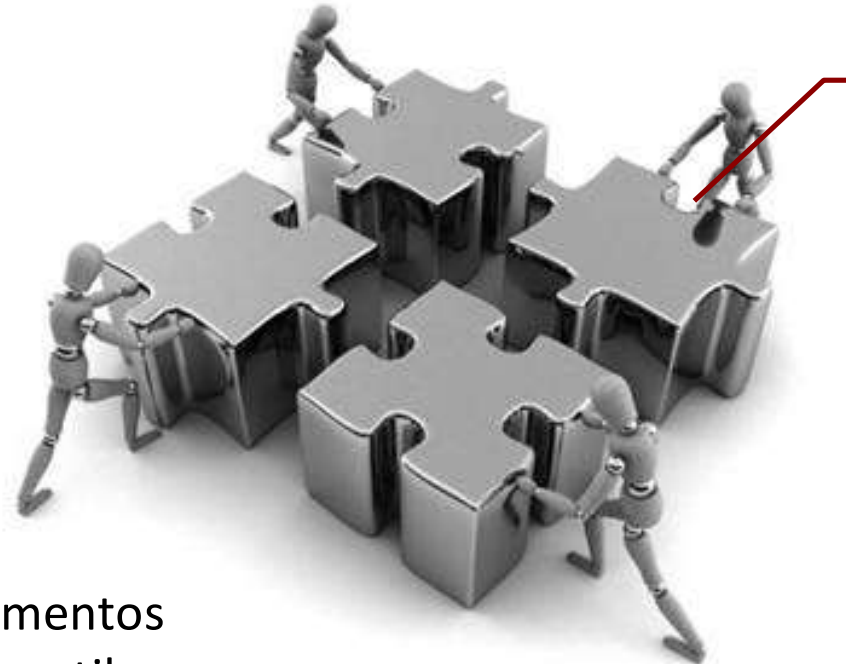
Diseño de datos



Transforma el modelo del dominio, obtenido del análisis, en estructuras de datos, objetos de datos, relaciones, etc.

# Diseño de Software - Tipos

---

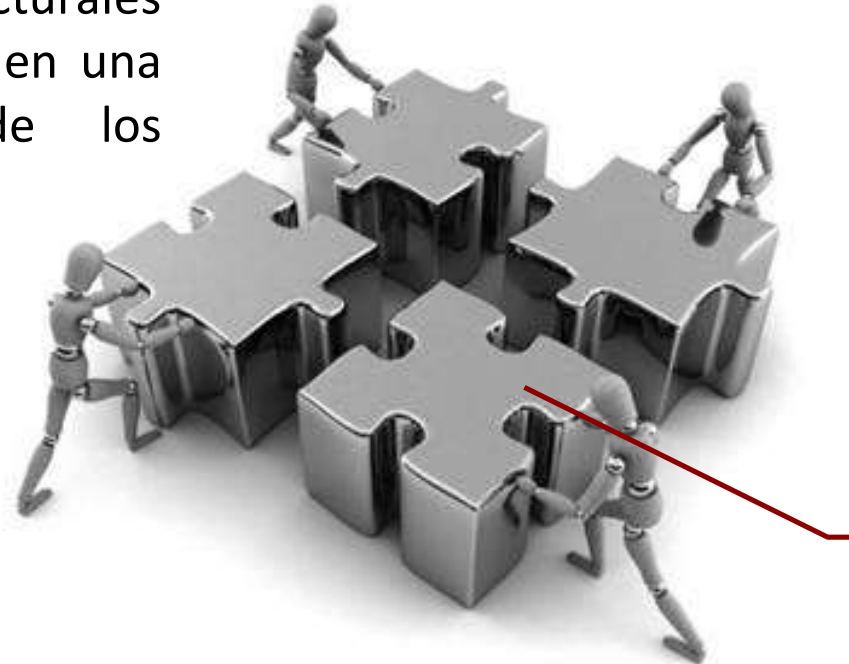


Diseño  
arquitectónico

Define la relación entre los elementos estructurales del software, los estilos arquitectónicos, patrones de diseño, etc.

# Diseño de Software - Tipos

Transforma los elementos estructurales de la arquitectura de software en una descripción procedimental de los componentes del software.

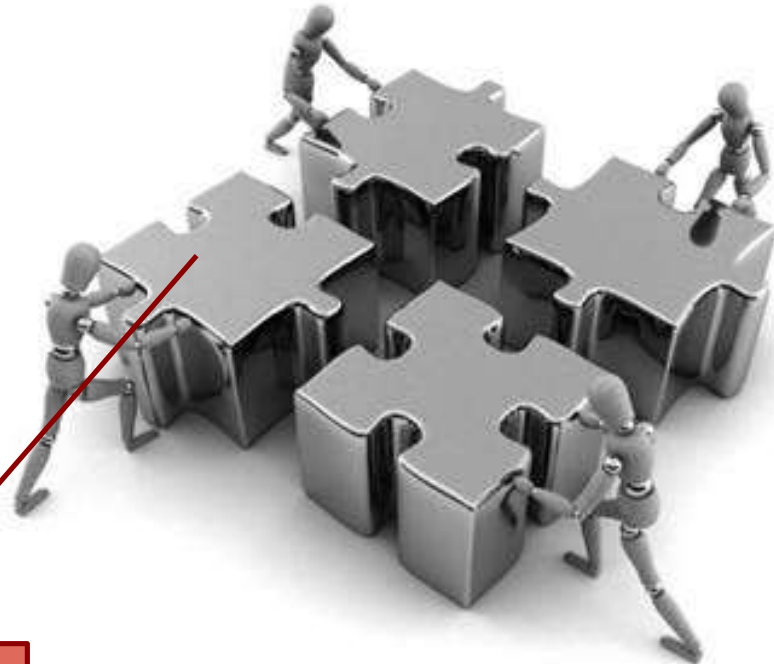


Diseño a nivel componentes

# Diseño de Software - Tipos

---

Describe la forma de comunicación dentro del mismo sistema, con otros sistemas, y con las personas.



Diseño de interfaz

# Diseño de Software – Características para su evaluación

---

- ❖ Deberá *implementar* todos los requerimientos explícitos del modelo de requerimientos, e *incorporar* todos los requerimientos implícitos que desea el cliente.
- ❖ Deberá ser una *guía* legible y comprensible para aquellos que generan código y para aquellos que dan soporte al software.
- ❖ Deberá proporcionar una imagen/visión completa del software (Compleitud).

# Criterios técnicos para un buen diseño

---

1. Deberá presentar una estructura arquitectónica que:  
*Se haya creado mediante patrones de diseño reconocibles, esté formada por componentes con buen diseño y se implemente en forma evolutiva.*
2. Deberá ser modular.
3. Deberá contener distintas representaciones.
4. Deberá conducir a estructuras de datos adecuadas y que procedan de patrones de datos reconocibles.

# Criterios técnicos para un buen diseño

---

5. Deberá conducir a componentes que presenten características funcionales independientes.
6. Deberá conducir a interfaces que reduzcan la complejidad de las conexiones entre los módulos y con el entorno externo.
7. Deberá derivarse mediante un método repetitivo y controlado por la información obtenida durante el análisis de los requisitos del software.
8. Deberá representarse por medio de una notación que comunique de manera eficaz su significado.

# Evolución del diseño de software

---

- ❖ Es un proceso continuo que abarca mas de seis décadas.
- ❖ Desde Programas modulares y refinamiento de estructuras de software, a enfoques orientado a objetos, orientado a modelos o a pruebas.
- ❖ Todos tienen características comunes:
  1. un mecanismo para traducir el modelo de requerimientos en una representación de diseño
  2. una notación para representar los componentes funcionales y sus interfaces
  3. heurísticas para refinamiento
  4. lineamientos para evaluación de calidad
- ❖ Sin importar el tipo de diseño, es necesario aplicar un conjunto de conceptos básicos.

# Conceptos de Diseño - Importancia

---

- ❖ ¿Qué criterios se usan para dividir el software en sus componentes individuales?
- ❖ ¿Cómo se extraen los detalles de la función o la estructura de datos de la representación conceptual del software?
- ❖ ¿Cuáles son los criterios uniformes que definen la calidad técnica de un diseño de software?

# Conceptos de Diseño



# Conceptos de Diseño

## Abstracción

- ❖ Permite concentrarse en un problema a un nivel de generalización sin tener en cuenta los detalles de bajo nivel.
- ❖ Tipos:
  - Procedimental: Secuencia “nombrada” de instrucciones que tienen una funcionalidad específica.
  - De datos: Colección “nombrada” de datos que definen un objeto real.

# Conceptos de Diseño

## Arquitectura

- ❖ Es la estructura general del software y las formas en que la estructura proporciona una integridad conceptual para un sistema.

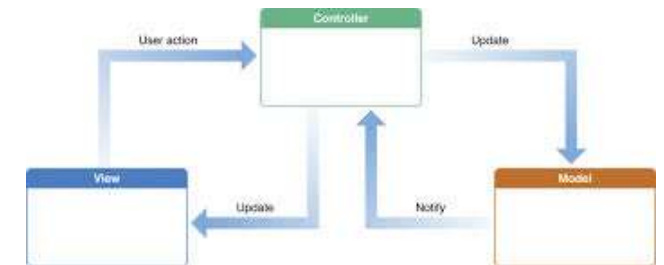
*Más adelante se estudiarán las arquitecturas con más detalle*



# Conceptos de Diseño

## Patrones

- ❖ Describen una estructura de diseño que resuelve un problema particular dentro de un contexto específico.
- ❖ Deben proporcionar una descripción que permita determinar si:
  - Es aplicable al trabajo
  - Se puede reutilizar
  - Puede servir como guía para desarrollar un patrón similar pero diferente en cuanto a la funcionalidad o estructura.



# Conceptos de Diseño

## Modularidad

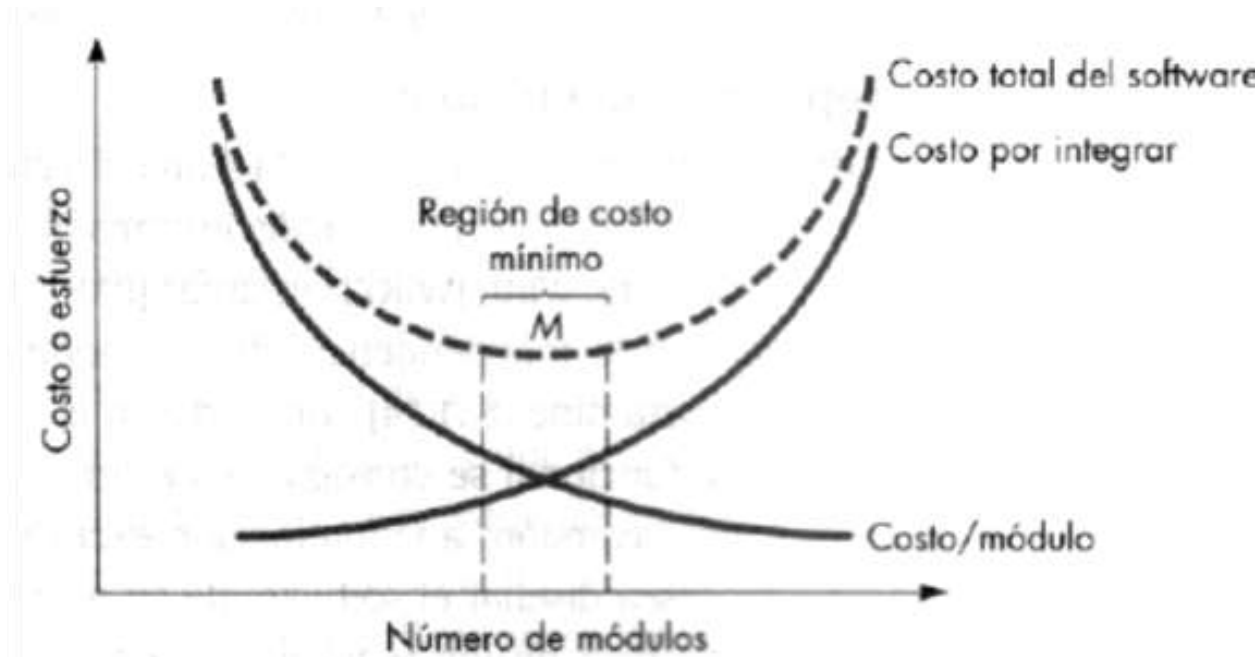
- ❖ El software se divide en componentes nombrados y abordados por separado, llamados frecuentemente módulos, que se integran para satisfacer los requisitos del problema.



# Conceptos de Diseño

## Modularidad

❖ ¿Cuántos módulos tiene que tener un programa?



# Conceptos de Diseño

---

## Ocultamiento de información

- ❖ La información que está dentro un módulo es inaccesible a otros que no la necesiten.



# Conceptos de Diseño

## Independencia funcional

❖ Modularidad + Abstracción + Ocultamiento de información



❖ Se busca la cohesión y el acoplamiento entre los módulos



# Conceptos de Diseño

## Independencia funcional

### ❖ Cohesión (Coherente)

*Medida de fuerza o relación funcional existente entre las sentencias o grupos de sentencias de un mismo módulo.*



Alta cohesión



Baja cohesión

# Conceptos de Diseño

## Independencia funcional

### ❖ Tipos de cohesión

Más alto



Funcional

*Cuando las sentencias de un módulo están relacionadas en el desarrollo de una única función.*

Comunicacional

*Se relaciona con los datos de entrada y salida.*

Procedimental

*Se relaciona con un intervalo de tiempo.*

Temporal

Lógica

*Se relaciona con tareas que no están relacionadas o tienen poca relación.*

Coincidental

Más bajo

# Conceptos de Diseño

## Independencia funcional

### ❖ Acoplamiento

*Medida de interconexión entre los módulos.*

*Punto donde se realiza la entrada o referencia y los datos que pasan a través de la interfaz.*



Bajo acoplamiento

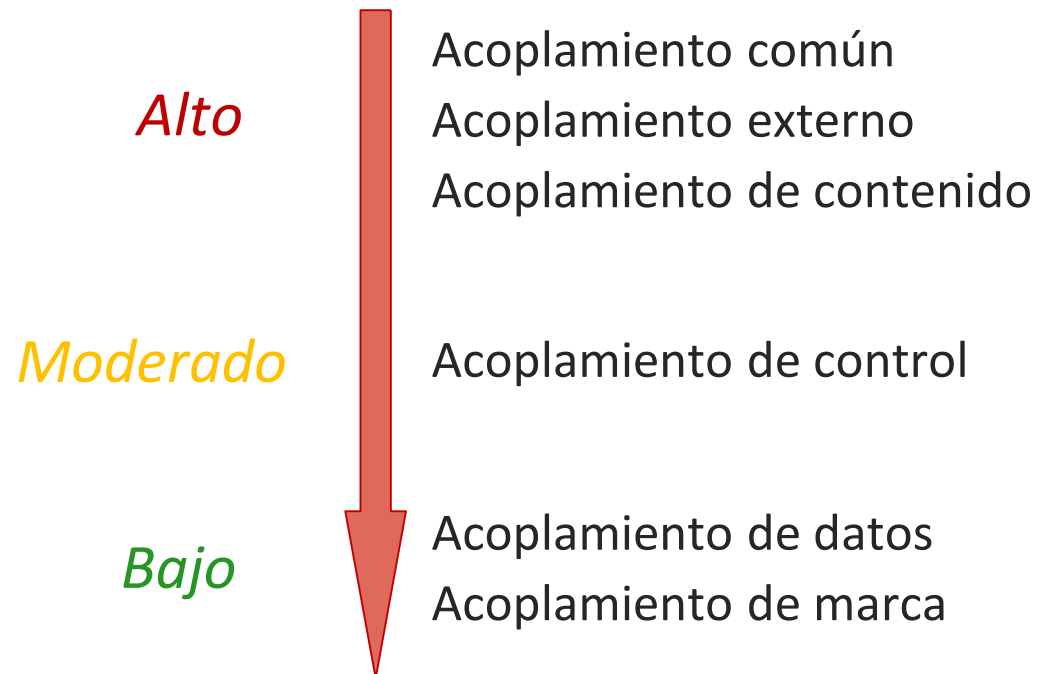


Alto acoplamiento

# Conceptos de Diseño

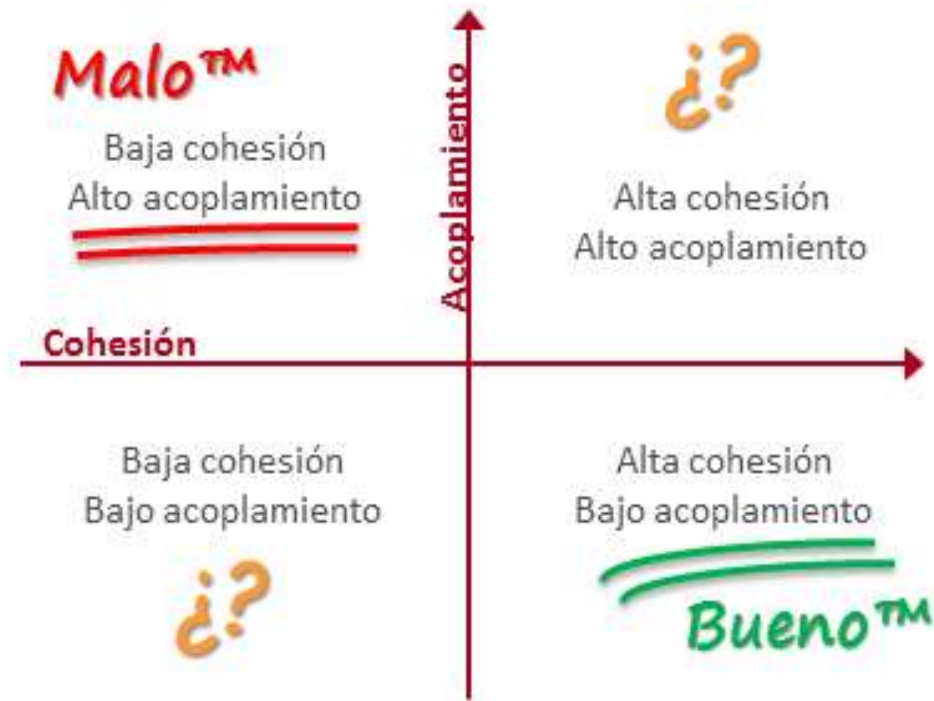
## Independencia funcional

### ❖ Niveles de acoplamiento



# Conceptos de Diseño

## Independencia funcional



# Cohesión y Acoplamiento: El Corazón del Diseño de Software

## Cohesión (La Fuerza Interna)

### ¿Qué es la Cohesión?

Mide la relación funcional y la fuerza de unión entre los elementos internos de un mismo módulo.

### El Objetivo: Alta Cohesión

Se busca que un módulo realice una única función (Cohesión Funcional) para facilitar su mantenimiento.

Definiendo la independencia funcional para software de alta calidad!

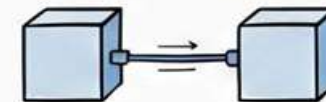
ALTA COHESIÓN  
+  
BAJO ACOPLAMIENTO  
=  
BUEN DISEÑO

## Acoplamiento (La Interconexión Externa)

### ¿Qué es el Acoplamiento?

Representa el grado de interdependencia o conexión entre los diferentes módulos de un sistema.

### El Objetivo: Bajo Acoplamiento



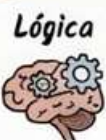
Bajo Acoplamiento (Ideal)



Alto Acoplamiento (Evitar)

Reduce el impacto de cambios, permitiendo que los módulos se modifiquen sin afectar al resto.

Escala de Cohesión (De Peor a Mejor):



### Matriz de Calidad para Evaluar el Diseño

| Atributo     | Estado Ideal | Resultado                            |
|--------------|--------------|--------------------------------------|
| Cohesión     | ALTA         | Diseño sólido y enfocado             |
| Acoplamiento | BAJO         | Sistema flexible y fácil de mantener |
| Relación     | "Bueno"      | Alta Cohesión + Bajo Acoplamiento    |

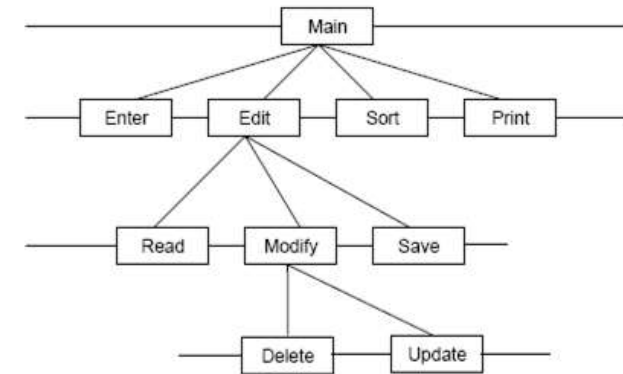
Escala de Acoplamiento (De Mejor a Peor):



# Conceptos de Diseño

## Refinamiento

- ❖ Se refina de manera sucesiva.
- ❖ La abstracción y el refinamiento son conceptos complementarios.
  - La abstracción permite especificar procedimientos y datos sin considerar detalles de grado menor.
  - El refinamiento ayuda a revelar los detalles de grado menor mientras se realiza el diseño.



# Conceptos de Diseño

## Refabricación

- ❖ Técnica de reorganización que simplifica el diseño de un componente sin cambiar su función o comportamiento.
- ❖ Llamado también rediseño (Refactoring).



# Principios del modelado de Diseño

---

1. El diseño debe poder rastrearse hasta el modelo de requerimientos.
2. Considerar siempre la arquitectura del sistema que se va a crear.
3. El diseño de los datos es tan importante como el diseño de funciones.
4. Las interfaces deben diseñarse con cuidado.
5. El diseño de interfaz debe ajustarse a las necesidades del usuario, haciendo énfasis en la facilidad de uso.
6. El diseño a nivel de componentes debe ser funcionalmente independiente.
7. Los componentes deben tener un bajo acoplamiento.
8. Las representaciones de diseño deben entenderse fácilmente.
9. El diseño debe desarrollarse de manera iterativa.
10. La creación de un modelo de diseño no impide metodología ágil.